

Programming quantum computers

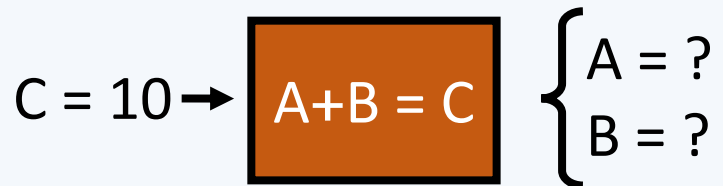
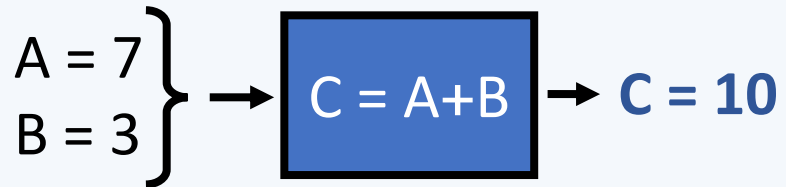
Irreversible and reversible computing

Classical computer programs are normally *irreversible*:

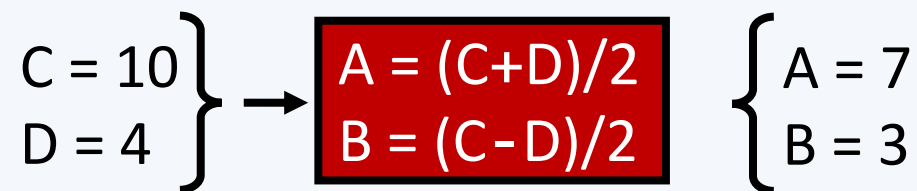
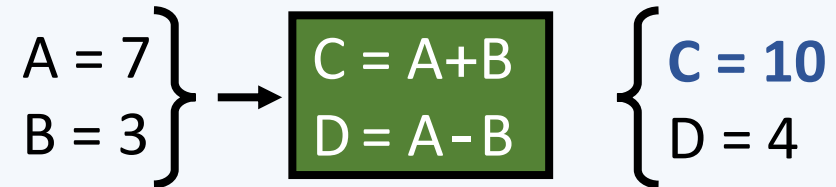
the input cannot be inferred from the output, *information is lost during computing*

- **Example:** Adding two numbers, $A + B \rightarrow C$

Irreversible



Reversible



Irreversible and reversible operations

Classical computing uses *irreversible* operations/computations on the basic information units, the bits

- Example, **OR**:
- From result “1” we do not know the input

X	Y	$X \vee Y$
0	0	0
0	1	1
1	0	1
1	1	1

Some operations are *reversible*, for example **NOT**: 0 -> 1
1 -> 0

Quantum operations are reversible

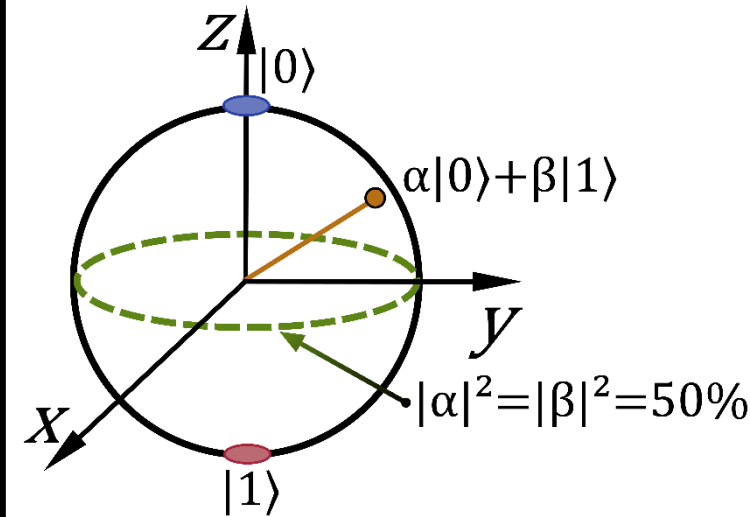
In quantum computing, **all** logical operations must be reversible!

- Follows from the rules of quantum mechanics
 - All operators must be *unitary*, $UU^* = I$
 - Keeps the probability $|\alpha|^2 + |\beta|^2 = 100\%$ for our qubit $\alpha|0\rangle + \beta|1\rangle$
- Using only reversible operations requires rethinking of algorithms
 - It is possible to do even classical computation with only reversible gates

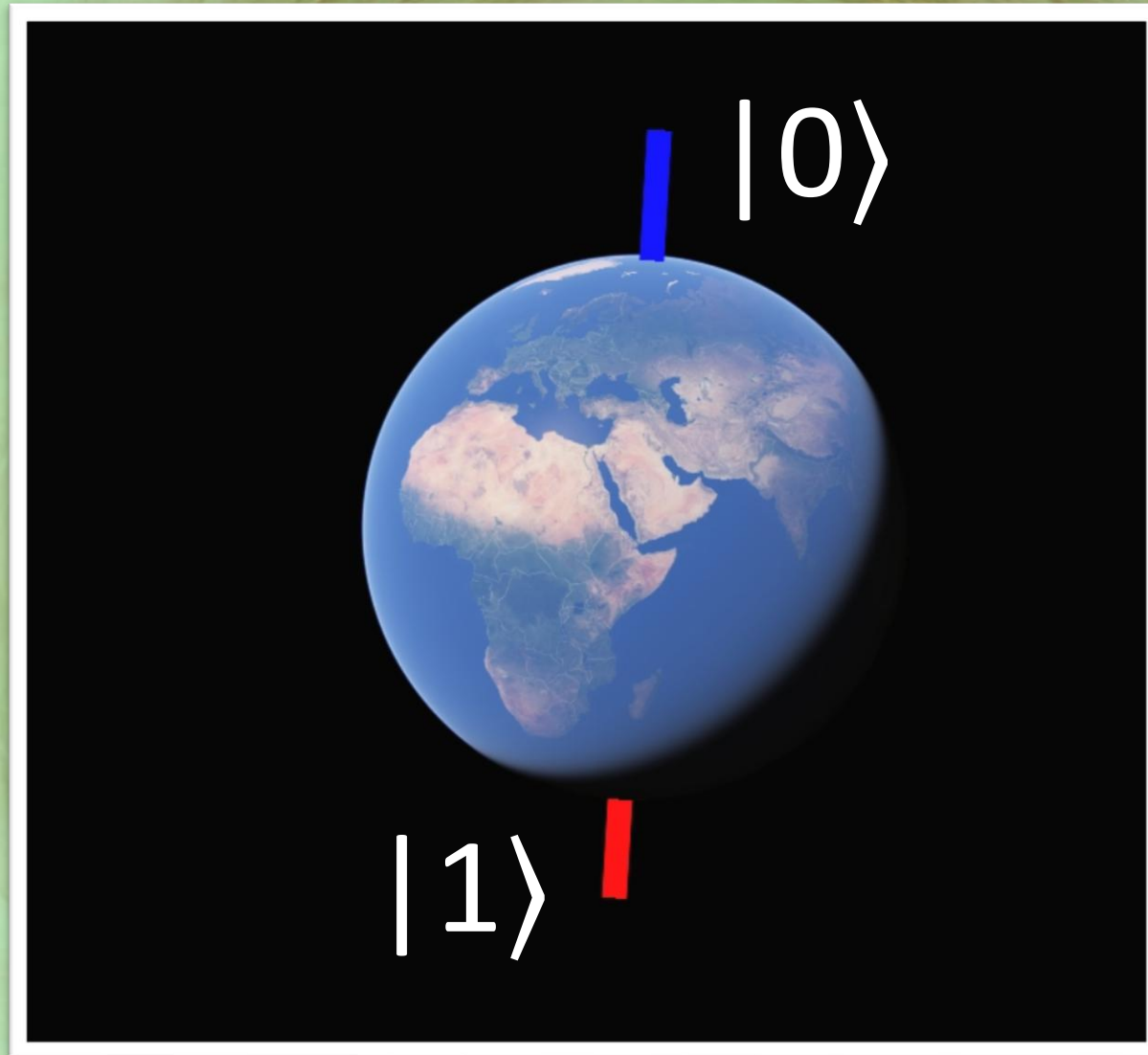
Bit / qubit manipulations

- A classical computer simply flips the states of the bits:
 $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \dots$
- The quantum bits, qubits, are **much more versatile**:
they can be something *in-between* $|0\rangle$ and $|1\rangle$:
 $\alpha|0\rangle + \beta|1\rangle$
- α and β are **complex numbers**: real and imaginary part
- Can be represented as **points on a sphere**
- In a quantum computer, the operations on the qubits
can and *have to* take advantage of this flexibility

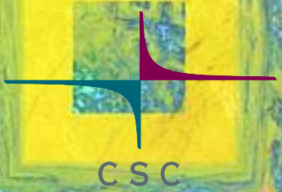
The Bloch Sphere



Bloch Sphere



Looking inside the box



- Maintaining superposition of the qubits is central to quantum computing
 - **When superposition is lost, the calculation is over**
- Superposition lost if the state of the qubit is **measured**, that is, **observed**



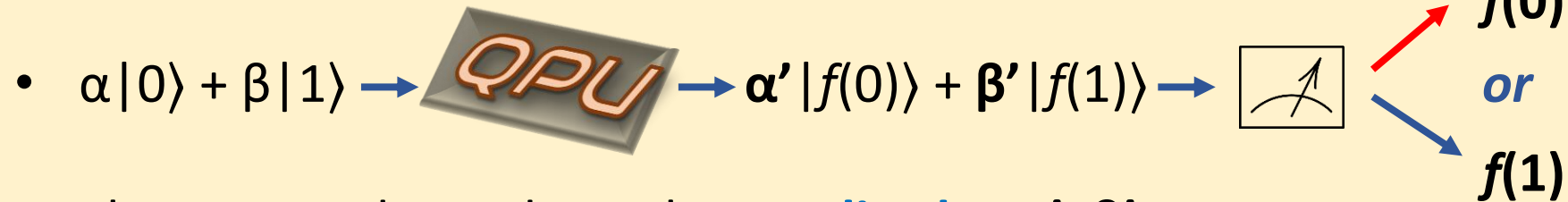
Image: Adobe Stock

- Measuring gives $|0\rangle$ *or* $|1\rangle$ also with the qubit in superposition of $|0\rangle$ *and* $|1\rangle$
 - After measuring $|0\rangle$ *or* $|1\rangle$ there is no way of going back, **the value sticks**
 - The values of the qubits can only be “printed out” once! At the end

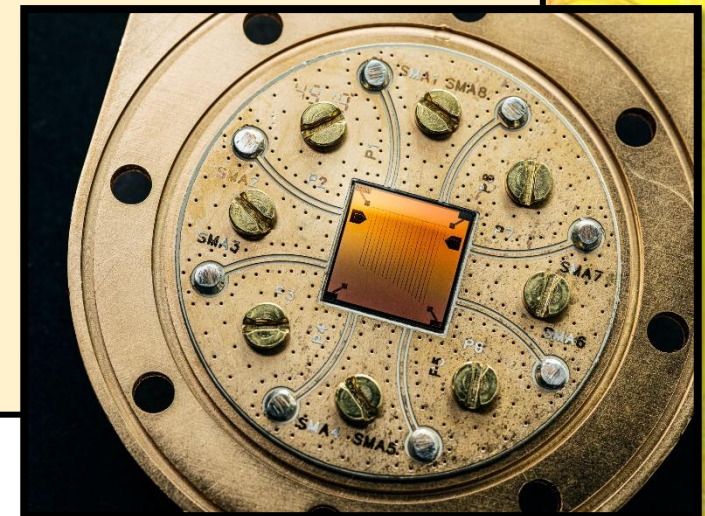
Measurement



- Even if **several inputs** can be processed at once, **only one answer** will emerge from the computer when you **measure** the result



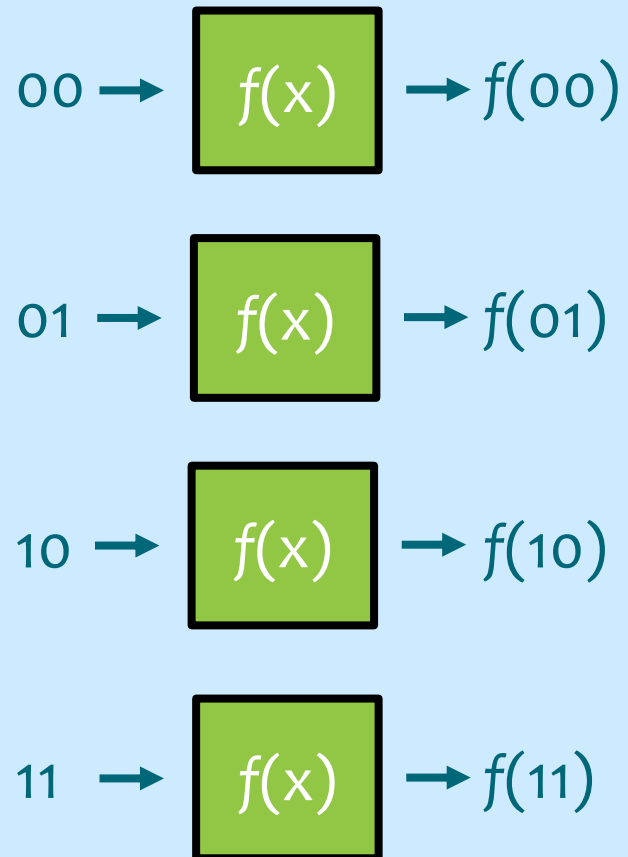
- The answer depends on the **amplitudes** α' , β'
- $|\text{amplitude}|^2 = \text{probability}$; $|\alpha|^2 + |\beta|^2 = 100\%$
- **A quantum computer is not deterministic**
 - In general, *different answers for the same input*
 - This really is a **feature, not a bug!**



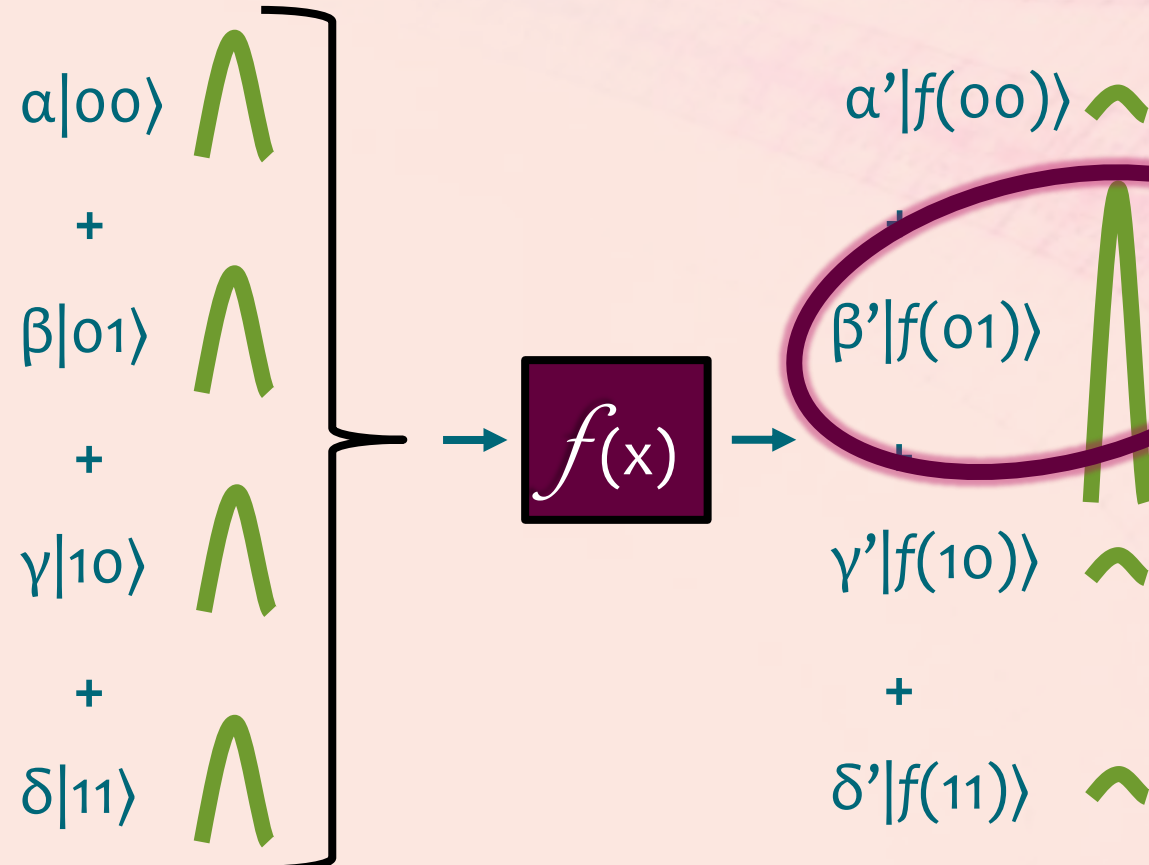
Quantum chip by IQM Quantum Computers

Parallelism from superposition

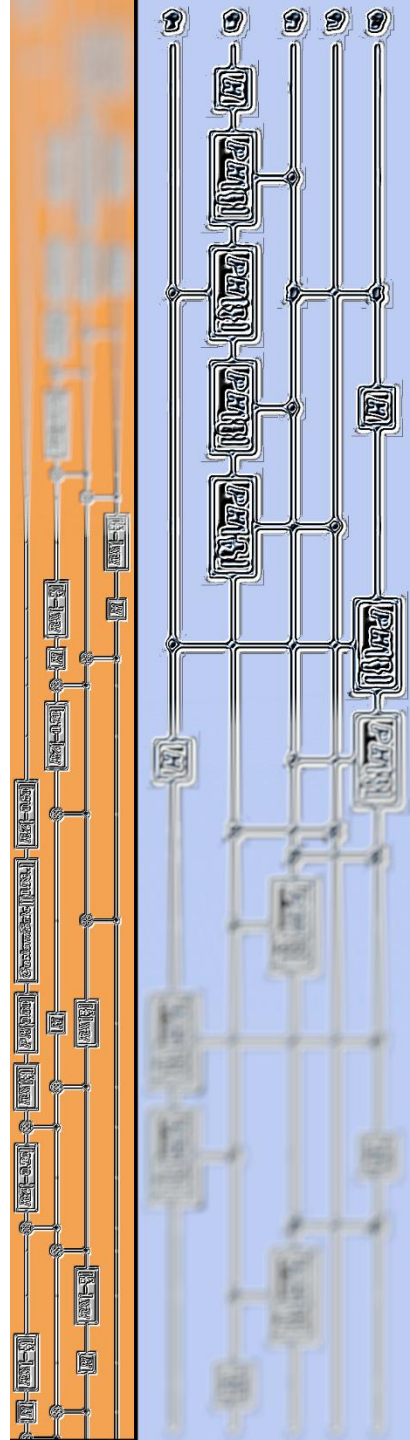
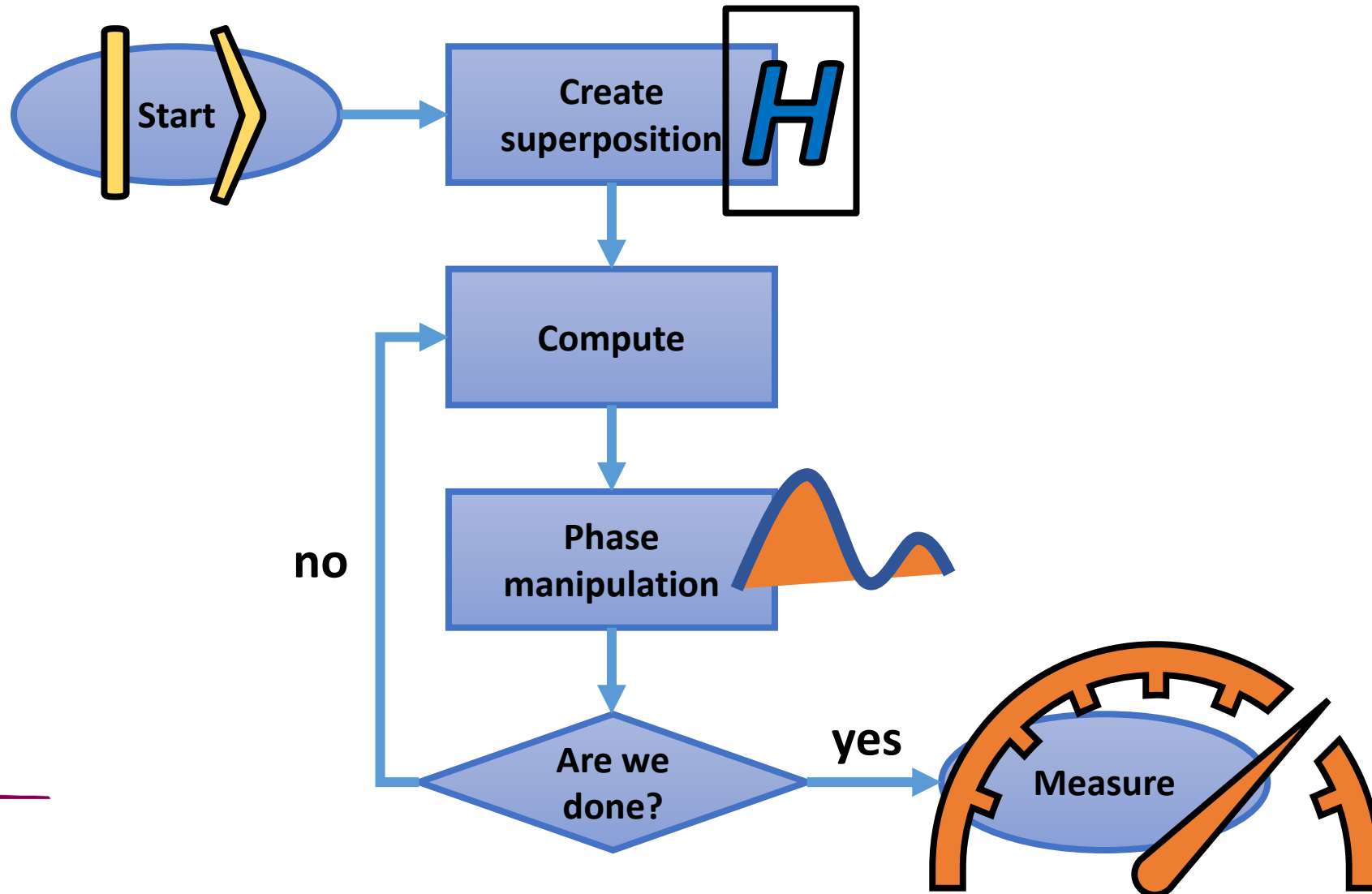
Classical computer



Quantum computer



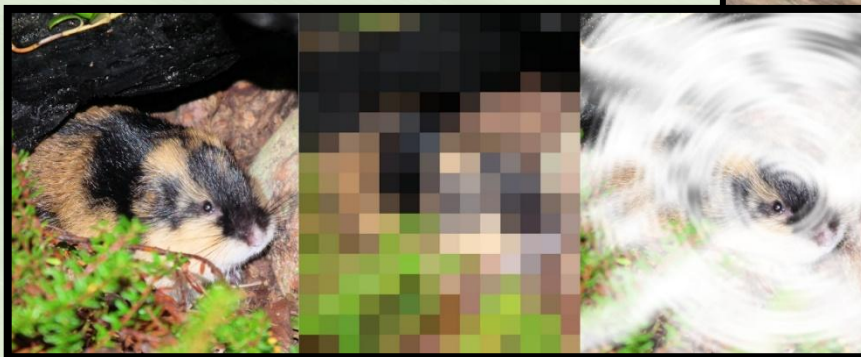
General quantum algorithm example



Disturbing noise

Qubits are **extremely delicate**: their states change due to outside influence, **noise** (like temperature): **superposition lost!**

- Affects the way algorithms need to be constructed in order to be efficient
- In **classical computing**, an inefficient program just runs for longer than necessary
- In **quantum computing**, a longer run-time leads to larger errors as well!



Kvanttisopuli: "Kohina" saa kvanttietokoneen tuottamaan hölynpölyvastauksia,
Johansson & Åström, *Tekniikan Maaailma* (online) 2020



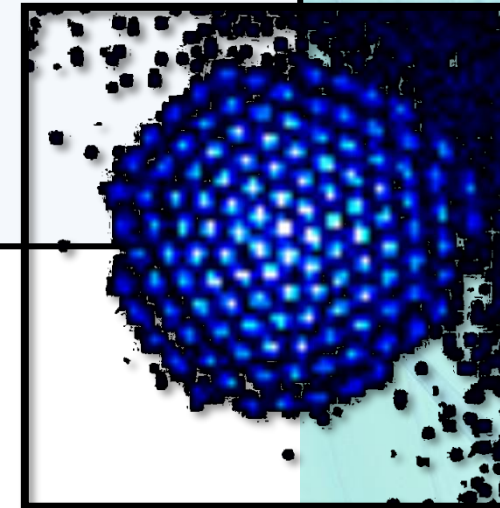
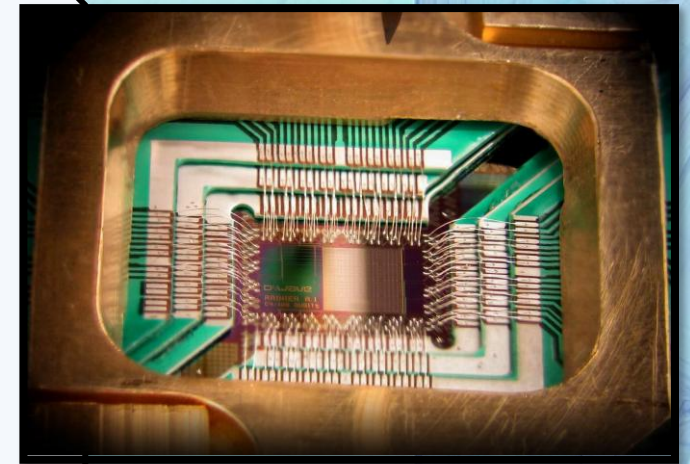
Flavours of quantum computers

Quantum computers are programmed in different ways, depending on how the “quantum” is implemented

Arguably, the main classes are

- Quantum annealers
- Quantum simulators
- Continuous variable quantum computers
- **Gate-based quantum computers**

Photo: D-Wave
CC BY 3.0



Trapped beryllium ions
Image: NIST

Quantum operations on qubits

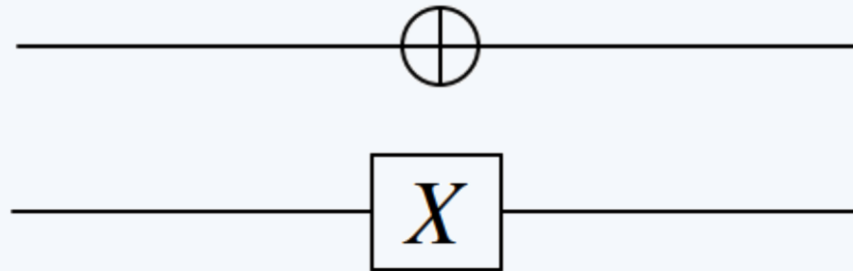
In computing using **quantum gates**, **operations** are represented using **linear algebra**

- States, qubits, are vectors $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$
- Operators are matrices, for example NOT $X := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
- Applying a quantum operation on a state means performing the linear algebra
 - For example NOT $|0\rangle$, that is, $X|0\rangle$

$$\begin{matrix} \text{NOT} & |0\rangle \\ \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & \begin{pmatrix} 1 \\ 0 \end{pmatrix} \end{matrix} = \begin{pmatrix} 0 + 0 \\ 1 + 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle$$

Quantum circuit diagrams

- Quantum algorithms are often shown as circuit diagrams
- One can use either symbols or names; for example for X



- A circuit diagram for NOT on $|0\rangle$ would then look like:



The Pauli operators

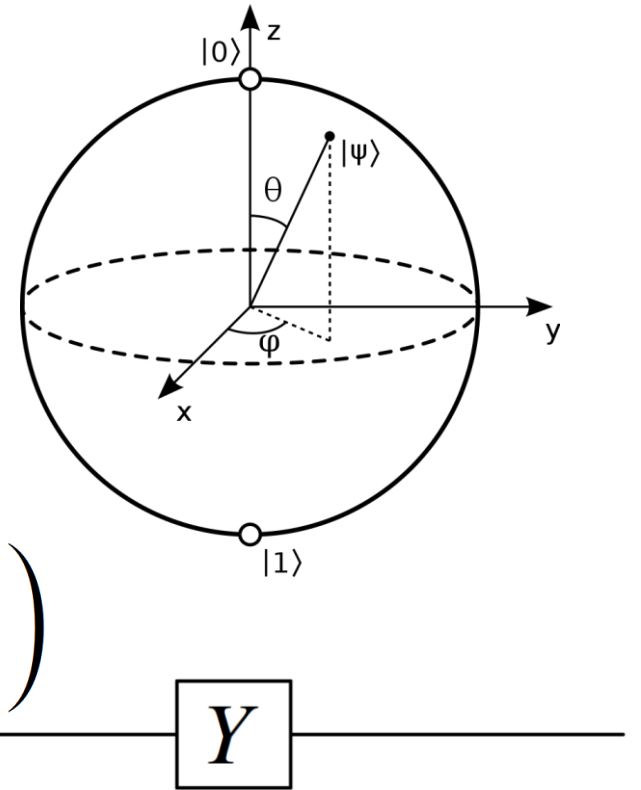
- Operators on single qubits are called **unary**

- The Pauli operators X, Y, Z**

- X, “bit flip”, NOT $X := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

- Y, rotation of the state vector around the y -axis $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
 - Example, $Y|1\rangle$

$$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 - i \\ 0 + 0 \end{pmatrix} = \begin{pmatrix} -i \\ 0 \end{pmatrix} = -i |0\rangle$$



Phase shift operators

- The Z operator, rotation around the z-axis

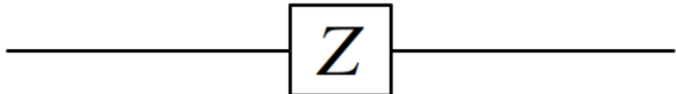
- Also called *phase flip*, as it flips the phase by π radians (180°)

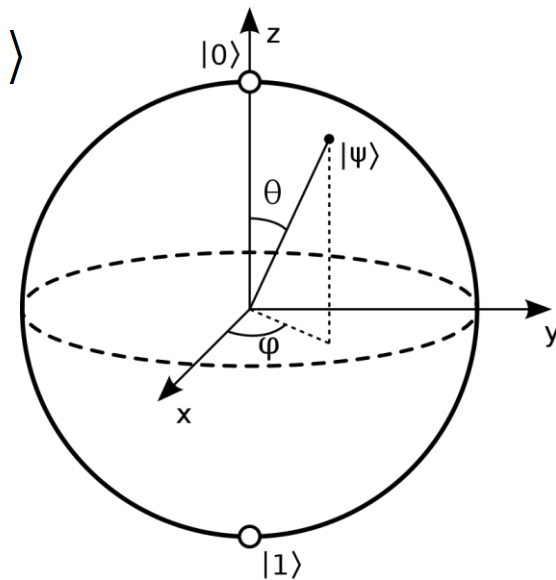
- Examples:

$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 + 0 \\ 0 + 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = (-1)^0 |0\rangle = |0\rangle$$

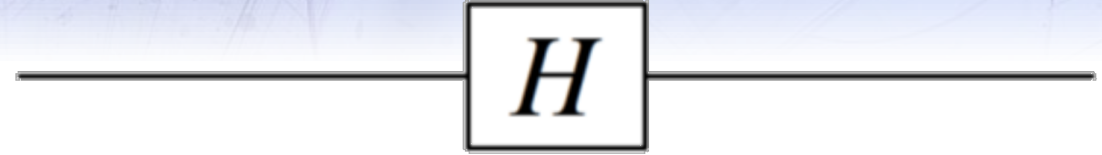
$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 + 0 \\ 0 - 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} = (-1)^1 |1\rangle = -|1\rangle$$

- General phase flip operator R_ϕ $R_\phi := \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix}$
- Z is just R_ϕ with $\phi = \pi$ ($e^{i\pi} = -1$)

$$Z := \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$




The Hadamard gate



- The **arch-quantum gate**, operator $H := \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
- Transforms a qubit from a specific state to a **superposition** of two states
- $H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1+0 \\ 1+0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$
- Note $\sqrt{2}$: The *square* of the amplitude is the probability of the state
 - The sum must be 1 (100%)
 - In general $\alpha|0\rangle + \beta|1\rangle$; $|\alpha|^2 + |\beta|^2 = 1$

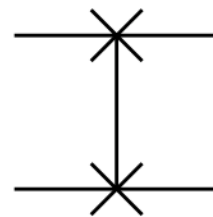
Binary operators

- With two qubits, the 4 different states are (by convention)

$$|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad |10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad |11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

- **Binary operators on 2 qubits are 4x4 matrices**

SWAP gate



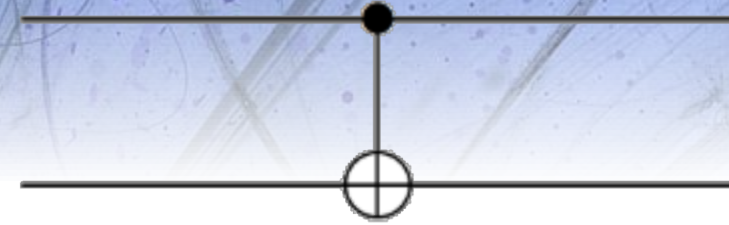
- The SWAP gate swaps the state of two qubits

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad |00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad |10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad |11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

$$SWAP := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{1} & 0 \\ 0 & \mathbf{1} & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\text{Example: } SWAP |01\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 + 0 + 0 + 0 \\ 0 + 0 + 0 + 0 \\ 0 + 1 + 0 + 0 \\ 0 + 0 + 0 + 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \mathbf{|10\rangle}$$

Controlled NOT, CNOT, CX



A **controlled** two-qubit gate uses one qubit to *control* the operation and the other as *target*

- If control = $|0\rangle$ then do nothing

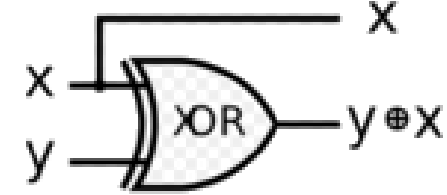
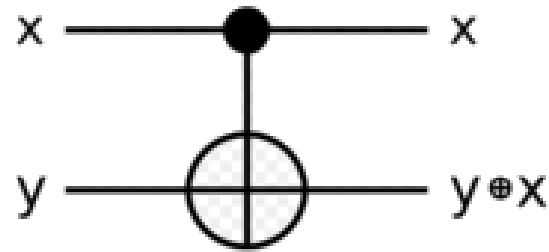
The controlled NOT, **CNOT** performs NOT on *target* if *control* = $|1\rangle$

$$CNOT := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$CNOT|10\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0+0+0+0 \\ 0+0+0+0 \\ 0+0+0+0 \\ 0+0+1+0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |11\rangle$$

- Analogous to classical eXclusive OR, **XOR**

CNOT vs XOR



input		output	
x	y	x	$y+x$
$ 0\rangle$	$ 0\rangle$	$ 0\rangle$	$ 0\rangle$
$ 0\rangle$	$ 1\rangle$	$ 0\rangle$	$ 1\rangle$
$ 1\rangle$	$ 0\rangle$	$ 1\rangle$	$ 1\rangle$
$ 1\rangle$	$ 1\rangle$	$ 1\rangle$	$ 0\rangle$

input		output	
x	y	x	$y+x$
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

All looks quite classical still.

But what if the input states are in superposition?

Operating on superposition states

When we have something else than the **basis states** $|0\rangle$ and $|1\rangle$, the quantum gates operate in a more complex manner

$$\alpha |0\rangle + \beta |1\rangle \longrightarrow \boxed{X} \longrightarrow \beta |0\rangle + \alpha |1\rangle$$

$$\alpha |0\rangle + \beta |1\rangle \longrightarrow \boxed{Z} \longrightarrow \alpha |0\rangle - \beta |1\rangle$$

$$\alpha |0\rangle + \beta |1\rangle \longrightarrow \boxed{H} \longrightarrow \alpha \frac{|0\rangle + |1\rangle}{\sqrt{2}} + \beta \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Entanglement

- Special kind of superposition

MAY 15, 1935

PHYSICAL REVIEW

VOLUME 47

Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?

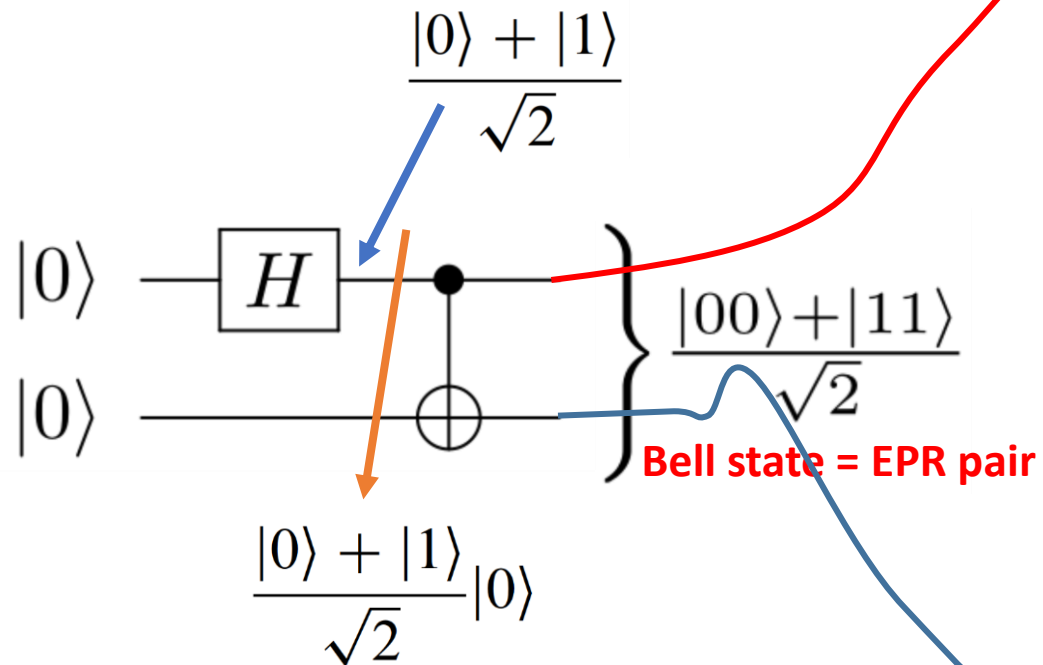
A. EINSTEIN, B. PODOLSKY AND N. ROSEN, *Institute for Advanced Study, Princeton, New Jersey*

(Received March 25, 1935)

- Two quantum states (say qubits) that are entangled experience the famous “*spooky action at a distance*” (orig. “*spukhafte Fernwirkung*”)
- In entangled superposition, the states of qubit A and B are unknown before measured
- After measuring *one* of the qubits, the state of the other is immediately set
- The superpositions of the two qubits are inseparable

H+CNOT = Entanglement

What if the *control* qubit for CNOT is in a superposition of $|0\rangle$ and $|1\rangle$?



- Equal superposition of **two out of four** possible two-qubit states
 - If qubit A = $|0\rangle$ also qubit B = $|0\rangle$
 - If qubit A = $|1\rangle$ also qubit B = $|1\rangle$
- **Before measuring**, we do not know either state, however!
- We *do* know they are equal -> **entanglement**

$$CNOT \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) = \frac{1}{\sqrt{2}} (CNOT|00\rangle + CNOT|10\rangle) = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

THE INSTITUTE FOR ADVANCED STUDY

PRINCETON, NEW JERSEY

July 5th, 1952

SCHOOL OF MATHEMATICS

Mr. Daniel M. Lipkin
4925 Rubicam Str.
Philadelphia 44, Pa.

Dear Mr. Lipkin:

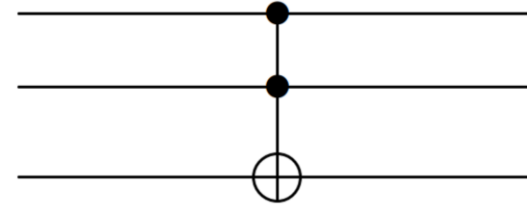
I too have many reasons to believe that the present quantum theory, in spite of its many successes, is far from the truth. This theory reminds me a little of the system of delusion of an exceedingly intelligent paranoiac concocted of incoherent elements of thought. As you also seem to believe I believe it impossible to get a real insight without satisfying from the start the principle of ^{general} relativity. I feel, however, by no means sure that my own approach is the right one.

Sincerely yours,

A. Einstein

Albert Einstein.

Toffoli, a ternary operator



- Ternary 3-qubit operators are represented by 8x8 matrices
- **Toffoli** = CCNOT, controlled-controlled-NOT
- If *both* control qubits are in state $|1\rangle$ then flip the third qubit

$$\bullet \text{ Ex: CCNOT} |110\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \color{red}{1} \\ 0 & 0 & 0 & 0 & 0 & 0 & \color{red}{1} & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |111\rangle$$

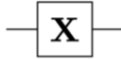
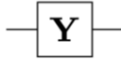
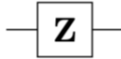
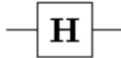
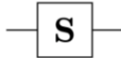
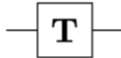
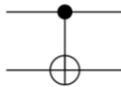
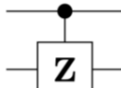
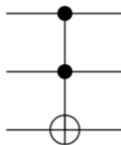
Quantum logic gates

A set of reversible logic operations to use for programming a quantum computer

- There are of course (infinitely) many more; however:

We do not need all of the gates, and most are not actually implemented on real quantum computers anyway!

- Real quantum computers have different gates, which might be conceptually more complex, but easier to implement in the real world
 - XX, CPHASE, CU1, U3, ...

Operator	Gate(s)		Matrix
Pauli-X (X)			$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)			$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)			$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)			$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Phase (S, P)			$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
$\pi/8$ (T)			$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
Controlled Not (CNOT, CX)			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
Controlled Z (CZ)			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP			$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Toffoli (CCNOT, CCX, TOFF)			$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$

Rxtreme / Wikipedia

Universal gate sets: NAND

- A **universal** set of gates can describe all other logic gates
- In **classical, irreversible** computing, the **NAND** gate, NOT-AND is universal (functionally complete)
 - All logical Boolean gates can be implemented by combining just NAND gates
 - Can require an exponential number of gates, though, so not actually efficient
- NAND is irreversible (multiple inputs possible from one output)
 - Thus, does not work for quantum computing

INPUT		OUTPUT
A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Universal gate sets: Toffoli

- Toffoli, CCNOT, is the **reversible** equivalent of NAND
- {Toffoli} is universal for classical computing
 - One can construct NAND from Toffolis
- Alone, Toffoli is *not* sufficient for **quantum** computing:
There is no way to create a superposition!
- **Universal quantum gate sets** include:
 - {Toffoli, H}
 - {CNOT, T, H}

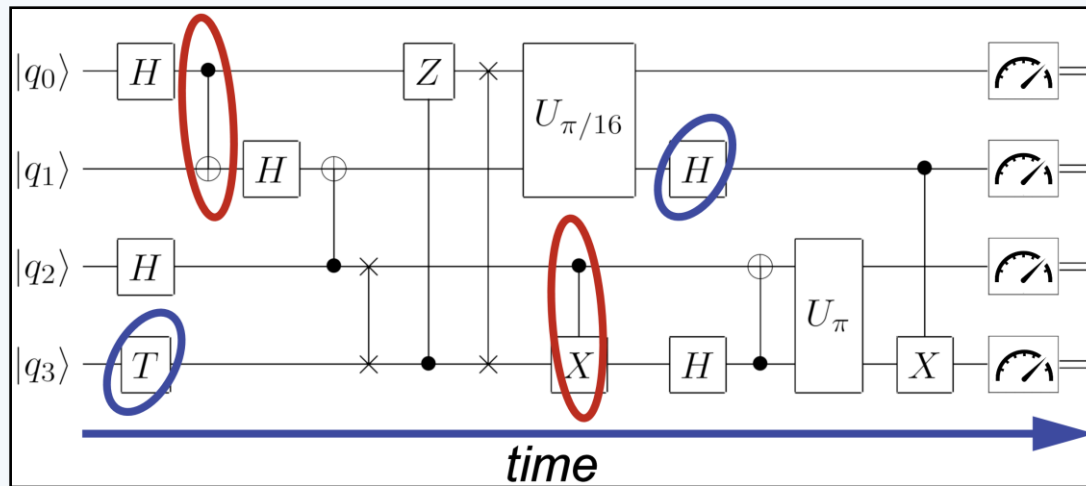
INPUT			OUTPUT		
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

Combining operations into algorithms

- The quantum gates (H, CNOT, SWAP, ...) are quite different from the commonly used classical operations (AND, OR, XOR, ...)
- Already this shows that programming quantum computers will be different
- In addition, to actually gain any advantage from a quantum computer, **we have to exploit superposition and entanglement!**
- This means that quantum algorithms have to be developed
- Problems need to be formulated so that they utilise quantum phenomena *explicitly*
- Many problems *cannot* be formulated so that there would be a quantum advantage
- Those that can, can see an **enormous** speed-up!

Combining operations into algorithms

- For now, quantum programming means writing rather **low-level code**
- **Quantum algorithms are built up from basic gate operations**



- **In the future/in the making:** libraries of usable quantum algorithms, more advanced compilers, FortranQ, *etc.*
- **Need to be developed so that they are ready when the big QPUs come!**

Ada King, Countess of Lovelace



Watercolour: Alfred Edward Chalon

First computer algorithms also written before the hardware

- Lovelace's Bernoulli-algorithm **published in 1843**
- Programmed for Charles Babbage's *Analytical Engine*

Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 *et seq.*)

Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data.										Working Variables.				Result Variables.			
						$1V_1$	$1V_2$	$1V_3$	$0V_4$	$0V_5$	$0V_6$	$0V_7$	$0V_8$	$0V_9$	$0V_{10}$	$0V_{11}$	$0V_{12}$	$0V_{13}$	$1V_{21}$	$1V_{22}$	$1V_{23}$	$0V_{24}$..	
						$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	
						0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
						0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
						1	2	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
						1	2	n											B_1 in a decimal fraction.	B_2 in a decimal fraction.	B_3 in a decimal fraction.	B_7	
1	$\times$	$1V_2 \times 1V_3$	$1V_4, 1V_5, 1V_6$	$\left\{ \begin{array}{l} 1V_2 = 1V_3 \\ 1V_3 = 1V_4 \\ 1V_4 = 2V_4 \\ 1V_5 = 2V_5 \\ 1V_6 = 2V_6 \end{array} \right.$	$= 2n$	...	2	n	$2n$	$2n$	$2n$												
2	-	$1V_4 - 1V_1$	$2V_4$	$\left\{ \begin{array}{l} 1V_4 = 2V_4 \\ 1V_5 = 2V_5 \\ 1V_6 = 2V_6 \end{array} \right.$	$= 2n - 1$	1	...	...	$2n - 1$														
3	+	$1V_5 + 1V_1$	$2V_5$	$\left\{ \begin{array}{l} 1V_5 = 2V_5 \\ 1V_6 = 2V_6 \end{array} \right.$	$= 2n + 1$	1	...	...	...	$2n + 1$													
4	+	$2V_5 + 2V_4$	$1V_{11}$	$\left\{ \begin{array}{l} 2V_5 = 0V_5 \\ 2V_4 = 0V_4 \end{array} \right.$	$= \frac{2n-1}{2}$	...	...	...	0	0													
5	+	$1V_{11} + 1V_2$	$2V_{11}$	$\left\{ \begin{array}{l} 1V_{11} = 2V_{11} \\ 1V_2 = 1V_2 \end{array} \right.$	$= \frac{1}{2} \cdot \frac{2n-1}{2}$	...	2	...	...														
6	-	$0V_{13} - 2V_{11}$	$1V_{13}$	$\left\{ \begin{array}{l} 2V_{11} = 0V_{11} \\ 0V_{13} = 1V_{13} \end{array} \right.$	$= -\frac{1}{2} \cdot \frac{2n-1}{2} = A_0$	...	...	...	...														
7	-	$1V_3 - 1V_1$	$1V_{10}$	$\left\{ \begin{array}{l} 1V_3 = 1V_3 \\ 1V_1 = 1V_1 \end{array} \right.$	$= n - 1 (= 3)$	1	...	n	...						n - 1								
													</										

The General Plan of
Mr. Babbage's Great Calculating Engine.

Levels of quantum programming

3. Ready-made libraries of common mathematical subroutines

- To be called from within a standard programming language
- “QBLAS”, Qiskit Aqua, Q# Libraries, ...
- New variable type `qubit` for existing languages?

```
logical function grand()  
  implicit none  
  qubit :: q0  
  q0 = (1.0,0.0,0.0,0.0)  
  q0 = q_H(q0)  
  grand = q_measure(q0)  
end function grand
```

2. Higher-level languages/SDKs for quantum programming

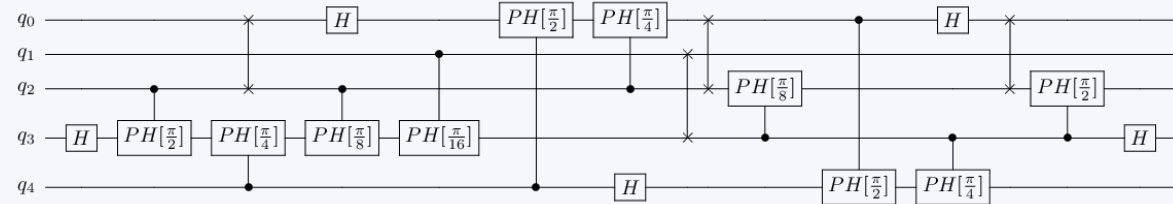
- Devise new quantum algorithms and apply existing algorithms to new problems
- Qiskit, PennyLane, Q#, Cirq, t|ket>, Silq, pyAQASM, Strawberry Fields, ...

1. Circuit-level assembly

- QIR, OpenQASM, AQASM, ...

0. Hardware-level coding

- Firmware, microcode, hardware-specific compilers, ...
- Not for general programming



Quantum programming challenges

1. Problem identification

- What exactly can/should QPUs compute?

2. Extracting advantage

- How to exploit superposition, entanglement, wave-function phases?

3. Working around limitations

- **Noise**, qubit connectivity, decoherence, ...

4. Hybrid HPC+QC computing

- Getting the best of both worlds

5. Very different programming paradigm

- All basic operations have to be **reversible!**
- **Only one measurement**, usually at the end
- Probabilistic, not deterministic
- **Uncomputing** necessary

$$F = \frac{q_1 q_2}{4\pi\epsilon_0 \epsilon r^2} \quad \Phi = \int \mathbf{B} \cos \alpha ds \quad f = \frac{v_m}{\lambda}$$

$$\vec{E} = \sum_{i=1}^N \vec{E}_i \quad \Psi(x)$$

$$v = \frac{1}{T}$$

$$R = \sigma T^4$$

$$x = A \cos(\omega t + \alpha) \quad \omega = 2\pi\nu \quad \Phi =$$

$$\sigma = 5.67 \cdot 10^{-8} \frac{W}{m^2 \cdot K^4}$$

$$R = \alpha \sigma T^4 \quad x = A_0 e^{-\beta' x}$$

$$\lambda_m = \frac{b}{T} \quad b = 2.9 \cdot 10^{-3} m$$

$$v_i = \frac{1}{T} \quad \omega = 2\pi\nu$$

$$\varphi = \arctan \frac{A_1 \sin \alpha_1 + A_2 \sin \alpha_2}{A_1 \cos \alpha_1 + A_2 \cos \alpha_2}$$

$$\Delta = \pm m\lambda_0, m = 0, 1, 2, \dots$$

$$f_p = \frac{f_0}{2\beta \sqrt{\omega_0^2 - \beta^2}} \quad W = \frac{1}{2} m A^2 \omega^2 \quad \xi =$$

$$M = Fv \quad \Delta \varphi = \frac{2\pi}{\lambda} \Delta x \quad \rho$$

$$\eta = \frac{1}{3} \rho \langle v \rangle \langle \lambda \rangle$$

$$U = \frac{1}{2} \frac{m}{M} RT$$

$$A = I \Delta \Phi$$

$$q = \frac{\Delta \Phi}{R}$$

$$D = \frac{1}{3} \langle v \rangle \langle \lambda \rangle \quad \Delta = L_2 - L_1 \quad \epsilon = \frac{q}{4\pi\epsilon_0 \epsilon r^2}$$

Programming challenges 75 years ago

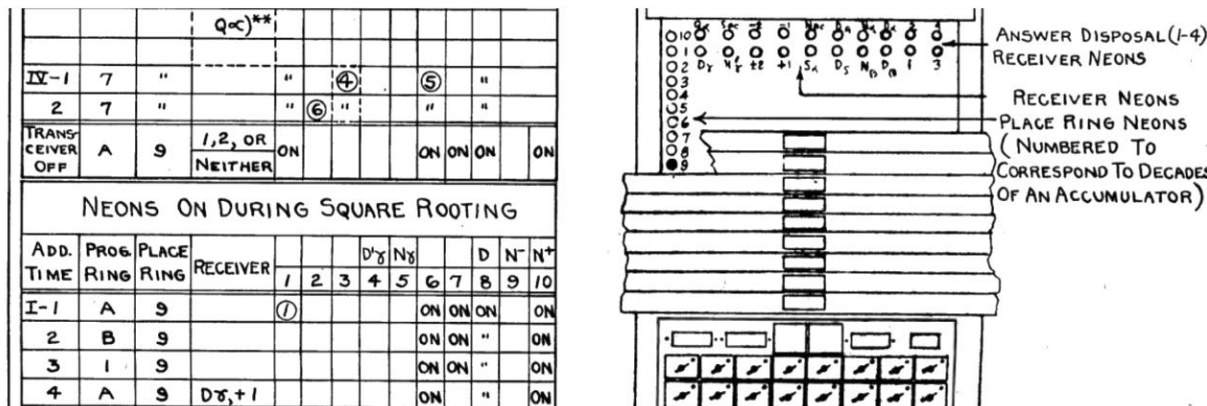
- **ENIAC (Electronic Numerical Integrator and Computer) 1946:**

7. Do not open d-c fuse cabinet with the d-c power turned on. This not only exposes a person to voltage differences of around 1500 volts but the person may be burned by flying pieces of molten fuse wire in case a fuse should blow.

We have perhaps become a bit spoilt!



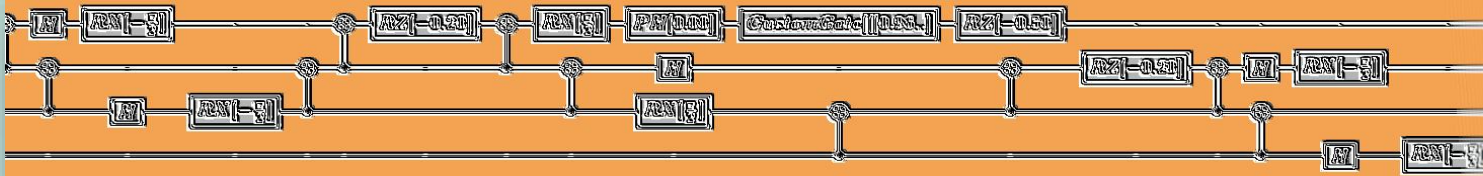
Picture: U.S. Army



Conclusions

CSC

- We already have quantum computers that can outperform the best supercomputers, *but not really for useful **computations***
- Need to develop algorithms that are **short enough**, so they complete before the quantum computer crashes!

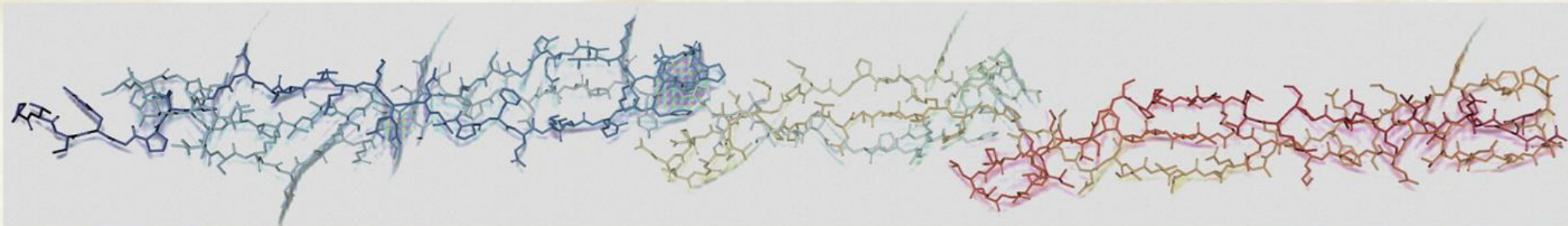


- **A truly multidisciplinary problem:** combine knowledge of mathematics, quantum physics, and computer science **with the application field!**
- Only with **both the software algorithms and hardware in place**, can the quantum revolution for computational modelling begin!

Conclusions

CSC

- Presently, **quantum computing is something of a dream come true for method developers**
 - Plenty of new areas to explore and use your creativity for **high-impact science!**



- **At some point, quantum computers and quantum computing will transform from being a subject *of* research to being a tool *for* research**
 - First incremental, then disruptive change in computational power
- **Let's start hacking!**

Hands-on 1

Programming quantum computers

Steps

1. Allocate quantum circuit – How many qubits your problem needs?
2. Apply quantum gates – These are instructions to quantum processing unit
3. Initialize backend - Choose your quantum computer
4. Transpile quantum circuit – Compile your program for target quantum computer
5. Run samples of quantum circuit – Execute your program on target quantum computer multiple times
6. Retrieve results -